TRIE: SEARCH & AUTO-COMPLETE

1. Introduction to Trie

A Trie (pronounced “try”) is a tree-based data structure used to store and search strings efficiently. It is also called a Prefix Tree because words with common prefixes share the same path in the tree.

Tries are commonly used in dictionaries, search engines, spell checkers, and auto-complete systems.

2. Structure of a Trie

Each node represents a character, and a path from the root represents a prefix. A special end-of-word marker identifies a complete word.

Example words: CAT, CAR, DOG

Root	Children / paths	Words formed
Root	C → A → T*	CAT
Root	C → A → R*	CAR
Root	D → O → G*	DOG

* indicates the end of a complete word.

Important points

- The root node does not represent a character.
- Each edge represents a character transition.
- Words with common prefixes share nodes.
- An end-of-word marker distinguishes a complete word from a prefix.

3. Trie Search

Trie Search checks whether a given complete word exists in a Trie. It follows the word’s characters one by one, starting at the root.

Algorithm

- Start from the root node.
- Move to the child corresponding to each character.
- If a required character node does not exist, the word is not present.
- If all characters exist, check the end-of-word marker.
- If the marker is set, the word exists; otherwise, only a prefix exists.

Example

Suppose the Trie stores CAT, CAR, and DOG:

- Search CAT → Found.
- Search CAR → Found.
- Search CAN → Not found.
- Search CA → Not a complete word, but it is a prefix of CAT and CAR.

Time complexity: O(L), where L is the length of the word. An iterative search uses O(1) auxiliary space, excluding the Trie.

4. Auto-Complete Using Trie

Auto-complete suggests complete words that begin with the prefix typed by a user. For example, entering CA may suggest CAT and CAR.

Algorithm

- Start from the root node.
- Follow the characters of the given prefix.
- If the prefix path does not exist, return no suggestions.
- If the prefix exists, traverse its subtree.
- Collect all words whose end-of-word markers are set.
- Return the collected words as suggestions.

Example suggestions

Prefix	Suggestions
CA	CAT, CAR, CARE, CART
CAR	CAR, CARE, CART
DO	DOG
Z	No suggestions

Suggestions may be displayed alphabetically or ranked by popularity, depending on the application.

5. Trie Search vs Auto-Complete

Trie Search	Auto-Complete
Checks whether a complete word exists.	Finds words that begin with a given prefix.
Usually returns true or false.	Returns a list of matching words.
Checks the end-of-word marker.	Traverses the prefix subtree to find complete words.
Example: search CAT.	Example: CA suggests CAT, CAR, CART.

6. Time Complexity of Auto-Complete

Let L be the length of the prefix, N the total number of nodes in the Trie, and K the total number of characters in the returned suggestions.

Operation	Time Complexity
Find the prefix	O(L)
Collect suggestions	Up to O(N)
Total	O(L + N) in the general case

The collection step can be optimized by stopping after a specified number of suggestions.

7. Applications of Trie

- Auto-complete in search engines.
- Word prediction in mobile keyboards.
- Spell checking and dictionary searches.
- Prefix-based searching.
- Contact-name suggestions.
- IP routing and text-processing applications.

8. Conclusion

A Trie is an efficient tree-based data structure for storing and searching strings. Trie Search checks whether a complete word exists, while Auto-Complete finds all complete words that match a given prefix. Both operations are useful for fast prefix-based searches.

Exam tip: Trie Search checks a complete word; Auto-Complete finds words beginning with a prefix.